Preliminaries

- Correctness and Loop Invariants
- Case Study: Sorting
- Case Study: Binary Search
- Case Study: KMP String Matching (X.ref week 7)
- Case Study: Quick Sort

Yanlin Zhang & Shangqi Lu | DSAA 2043 Fall 2026

Midterm Exam

香港科技大学(广州)
THE HONG KONG UNIVERSITY OF SCIENCE AND TECHNOLOGY (GUANGZHOU)

- **Time: 14:00–15:30, Sun, Oct. 25, 2026 (updated)**
- Location: TBA (Lab room)
- Format: On computer
  - solve algorithmic problems by writing **correct** code
  - Pseudocode is accepted for partial credit, typically around **50–70%**, depending on its correctness and completeness.
- Preparation:
  - Having a good understanding of algorithms covered in class
  - Learn to write code without LLM (i.e., no copilot, no cursor, etc.)
  - Know basic python grammar such as use *self* in a python class.

Algorithm v.s. Correct Algorithm

- An algorithm passes: all provided test cases, and 1,000,000 random tests. Is it correct?
  - Just vote: **Yes / No / Not sure**.
- Algorithm vs. Correct Algorithm:

Algorithm

A finite, well-defined, executable procedure.

Correctness

A property we prove *about* an algorithm. An algorithm may be incorrect, but it is still an algorithm.

Correctness

- **Definition.** An algorithm is *correct* if, for every valid input, it terminates and produces an output that satisfies the problem specification.
- This definition has three essential components:
  - universality (for all inputs),
  - Termination
  - specification satisfaction.

A Common but Invalid Argument

香港科技大学(广州)
THE HONG KONG UNIVERSITY OF SCIENCE AND TECHNOLOGY (GUANGZHOU)

Claim (incorrect)

"The algorithm works because I tested it on many random inputs."

Failure analysis:

- The input space is typically huge (often effectively infinite)
- Testing covers only finitely many cases
- Bugs often appear only in corner cases (boundaries, duplicates, off-by-one)

What Proofs Protect You From

- Passing all tests but still being wrong
- Infinite loops you didn’t anticipate
- Boundary cases you forgot
- “Seems correct” intuition

Correctness of Iterative Algorithms

- Most algorithms in this course are: iterative, state-based (arrays, indices, pointers), gradually transforming the input into the output.
  →We must therefore reason about intermediate states, not just the final result.
- Central Question:
  – During the execution of a loop, what is guaranteed to be correct so far?

Answer

A loop invariant captures these guarantees at iteration boundaries.

Example: Proving the Correctness of FindMax


```
Algorithm FindMax(A)
Input: An array A with n elements
Output: The maximum value in A

1. max_so_far ← A[1]
2. for i ← 2 to length(A) do
3.     if A[i] > max_so_far then
4.         max_so_far ← A[i]
5.     end if
6. end for
7. return max_so_far
```

Proof of Correctness

Goal: Showing that max_so_far stores the maximum value of A.

1. Before the Loop Starts (Initialization)

- Before entering the loop, max_so_far = A[1]. Since we've only seen one element, this is correct.

2. During Each Iteration (Maintenance)

- If A[i] > max_so_far, we update max_so_far = A[i], ensuring it holds the maximum seen so far.
- Otherwise, max_so_far remains unchanged, which is still correct.

3. After the Loop Ends (Termination)

- At the end, max_so_far contains the maximum of all A[1] to A[n].

Loop Invariant

- A loop invariant is a property or condition that holds true before and after each iteration of a loop.
- Purpose:
  - To show that an algorithm maintains a specific condition throughout its execution.
  - To help prove that the algorithm works correctly (via initialization, maintenance, and termination).
- The way that we prove FindMax correct is Loop Invariant.
  - Loop invariant: max_so_far holds the maximum value among A[1:i].

Note

The invariant is required to hold at iteration boundaries (end of iterations), not during the loop body.`

Structure of a Loop Invariant Proof

- Initialization: Show that the invariant holds **before** the first iteration (base case).
- Maintenance: Assuming the invariant holds at the beginning of any iteration, **prove that it still holds** after executing the loop body.
- Termination: When the loop terminates, the invariant (plus the loop's exit condition) gives a useful property that helps prove the algorithm's correctness.

Warning

Omitting any component invalidates the proof.

FYI. The three steps is introduced in CLRS. In other books, you may find Establishment (i.e. Initialization), Preservation (i.e. Maintenance), Postcondition and Termination: Postcondition ensures the final goal is achieved if the loop stops; Termination guarantees that the loop will stop.

A Subtle but Common Failure

Claimed invariant (incorrect)

"The invariant holds because it is true after one iteration."

Failure analysis:

- This only argues maintenance
- Initialization is never established
- No anchor to the execution start

Bottom line

Maintenance alone is never sufficient.

How to Interpret an Invariant

Read it like this

"When the loop index is i, this part of the algorithm's goal has already been achieved."

It describes:

- What is already correct
- What remains to be done
- How correctness progresses with i

Not all invariants are equally useful. An invariant can be:

- Too weak — easy to maintain, but insufficient to imply correctness
- Too strong — implies correctness, but hard or impossible to maintain
- Just strong enough

Weak Invariant Example

Weak invariant (sorting)

"Some elements are already in the correct position."

Why this fails:

- Does not specify which elements
- Does not depend on i
- Cannot imply full sortedness at termination

Overly Strong Invariant Example

Overly strong invariant

"At the end of iteration i, the entire array is sorted."

Why this fails:

- False during early iterations
- Cannot be established during initialization
- Breaks the maintenance step

The “Just-Right” Invariant

A good invariant typically:

- Refers explicitly to loop index i
- Fixes a prefix, suffix, or subset
- Grows monotonically as i increases

Such invariants are:

- Provable
- Maintainable
- Sufficient for correctness

How to find invariant

Treat invariant design as a mechanical procedure:

- Write the final correctness condition (postcondition)
- Identify what is not yet guaranteed early in the loop
- Remove that part
- Index what remains by the loop variable(s)

Typical failure mode:

- Start from intuition
- Jump directly to the final result
- Skip intermediate guarantees

Takeaway

These steps give you a practical strategy for finding loop invariants

Standard Loop Invariant Pattern (Array Iteration)

香港科技大学(广州)
THE HONG KONG UNIVERSITY OF SCIENCE AND TECHNOLOGY (GUANGZHOU)

Template

"At the end of iteration i, the substructure [explicit range depending on i] satisfies [precise property]."

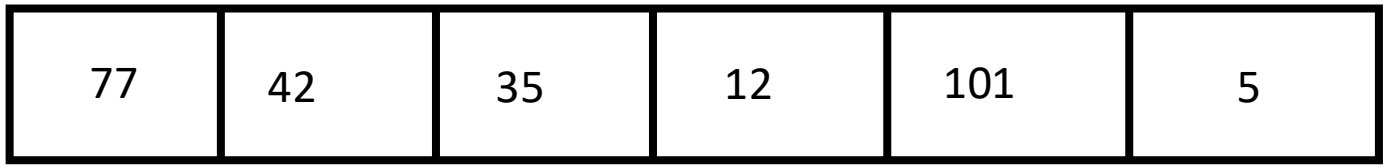
Rule of thumb

Any invariant not fitting this template should be treated with suspicion.

Case study: Sorting

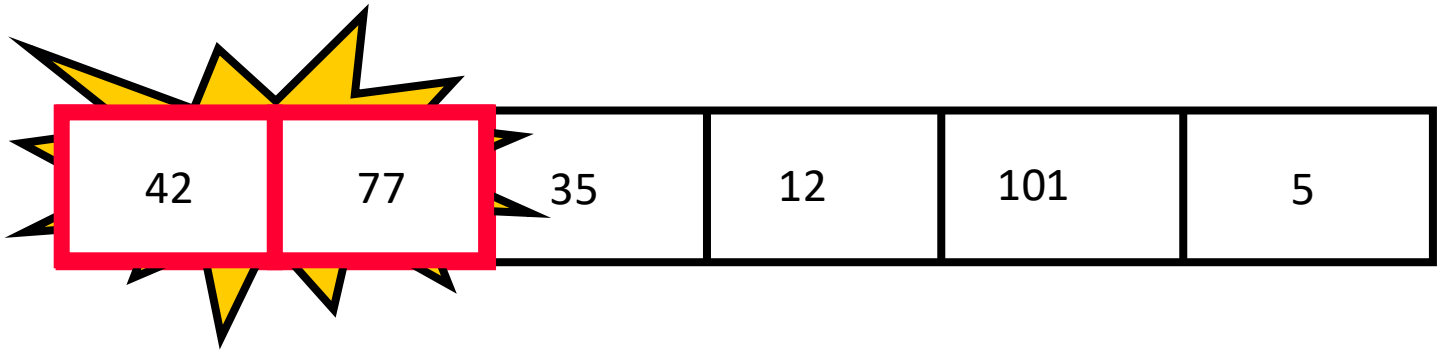
Bubble Sort: "Bubbling Up" the Largest Element

- Traverse a collection of elements
- Move from the front to the end
- “Bubble” the largest value to the end using pair-wise comparisons and swapping



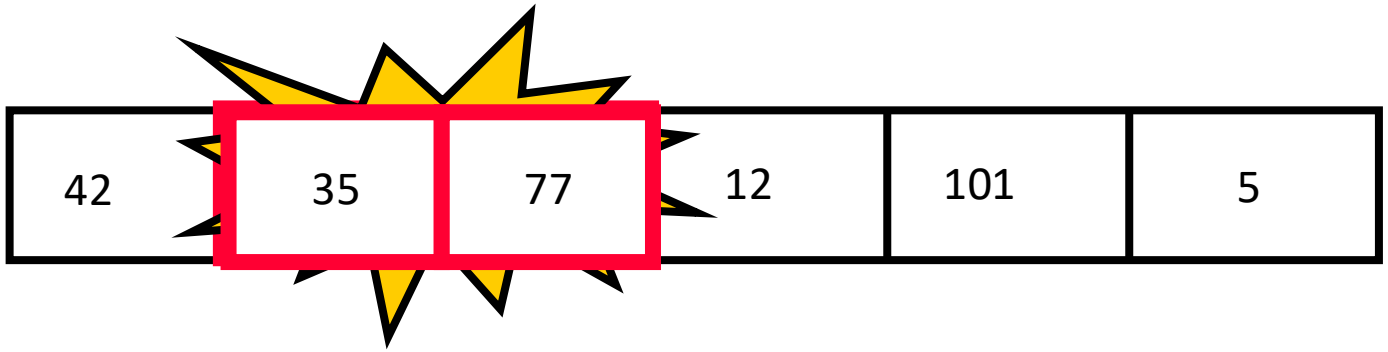
Bubble Sort: "Bubbling Up" the Largest Element

- Traverse a collection of elements
- Move from the front to the end
- “Bubble” the largest value to the end using pair-wise comparisons and swapping



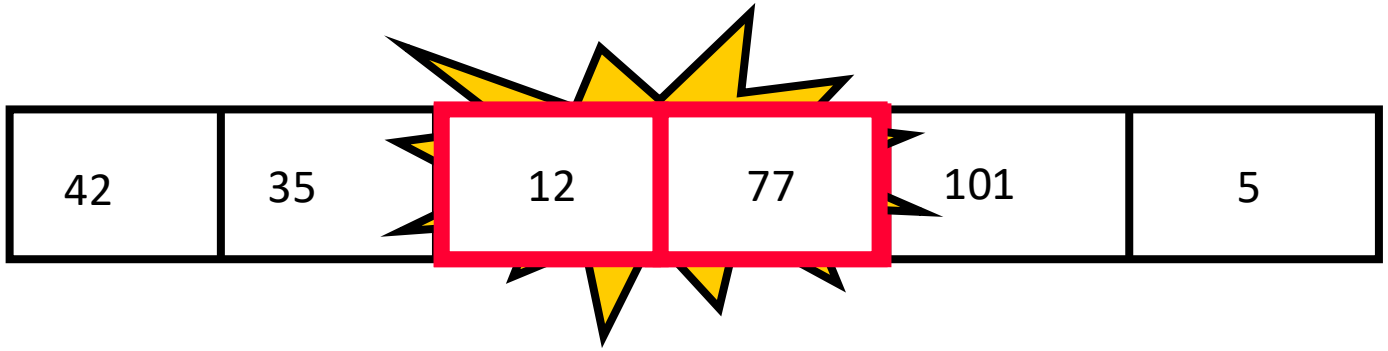
Bubble Sort: "Bubbling Up" the Largest Element

- Traverse a collection of elements
- Move from the front to the end
- “Bubble” the largest value to the end using pair-wise comparisons and swapping



Bubble Sort: "Bubbling Up" the Largest Element

- Traverse a collection of elements
- Move from the front to the end
- “Bubble” the largest value to the end using pair-wise comparisons and swapping



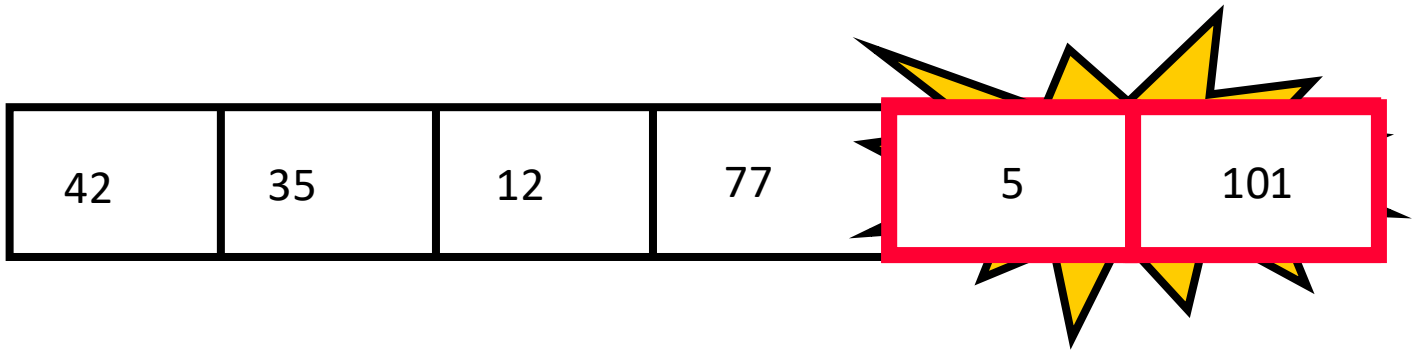
Bubble Sort: "Bubbling Up" the Largest Element

- Traverse a collection of elements
- Move from the front to the end
- “Bubble” the largest value to the end using pair-wise comparisons and swapping

No need to swap

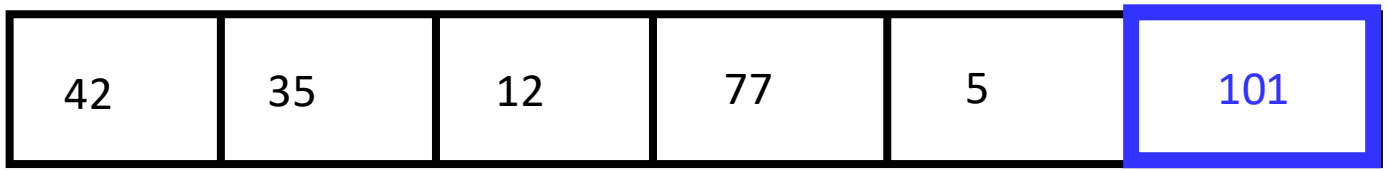
Bubble Sort: "Bubbling Up" the Largest Element

- Traverse a collection of elements
- Move from the front to the end
- “Bubble” the largest value to the end using pair-wise comparisons and swapping



Bubble Sort: "Bubbling Up" the Largest Element

- Traverse a collection of elements
- Move from the front to the end
- “Bubble” the largest value to the end using pair-wise comparisons and swapping



Largest value correctly placed

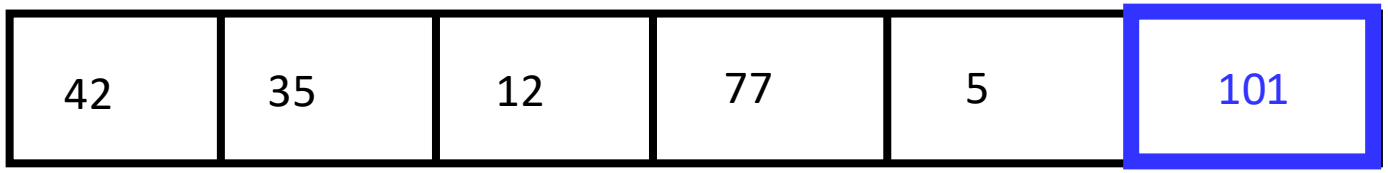
The “Bubble Up” Algorithm


```
index ← 1
last_compare_at ← n – 1

while index < last_compare_at+1
   if(A[index] > A[index + 1]) then
      Swap(A[index], A[index + 1])
   end if
   index ← index + 1
end while
```

Items of Interest

- Notice that only the largest value is correctly placed
- All other values are still out of order
- So we need to repeat this process

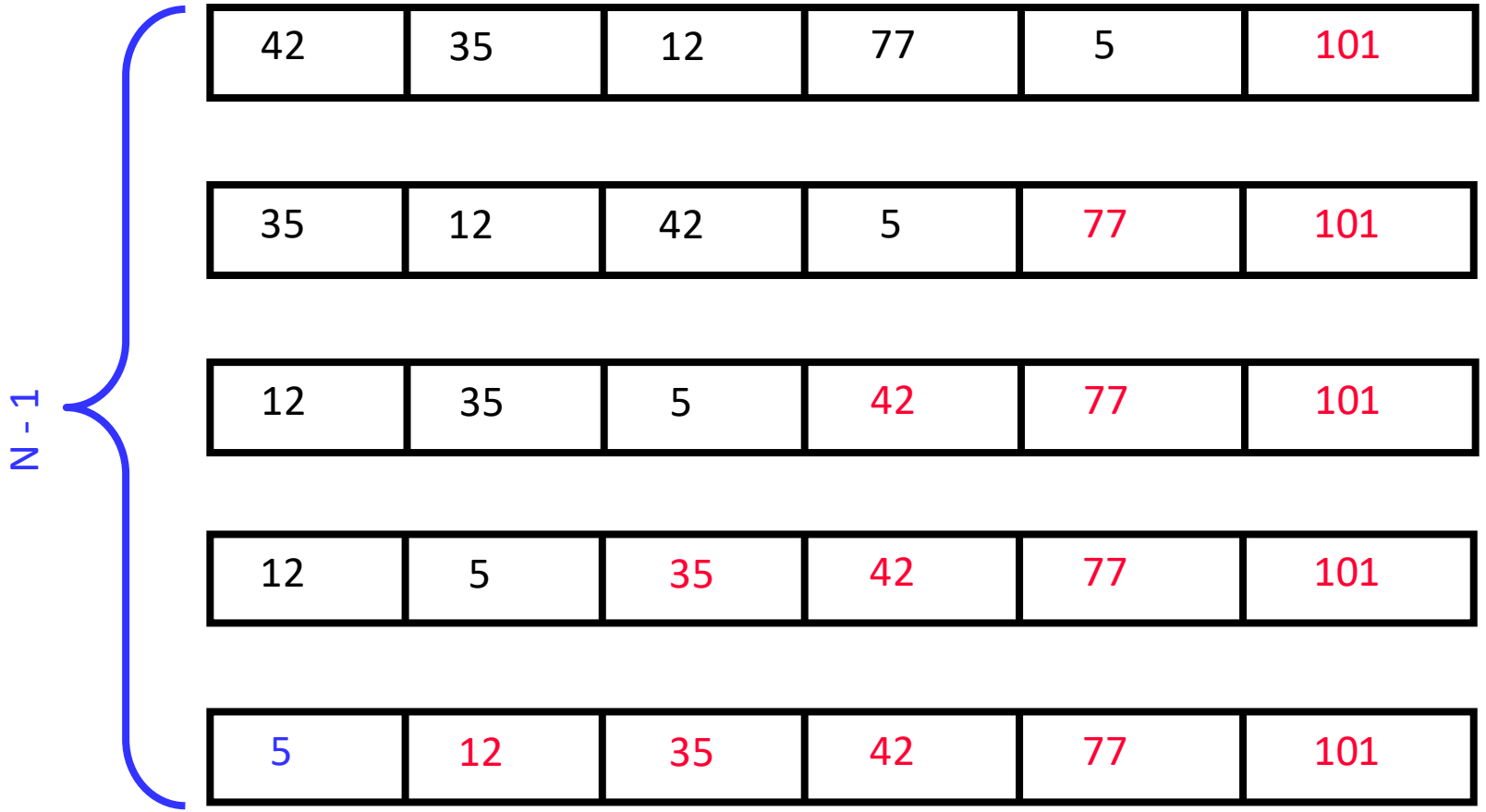


Largest value correctly placed

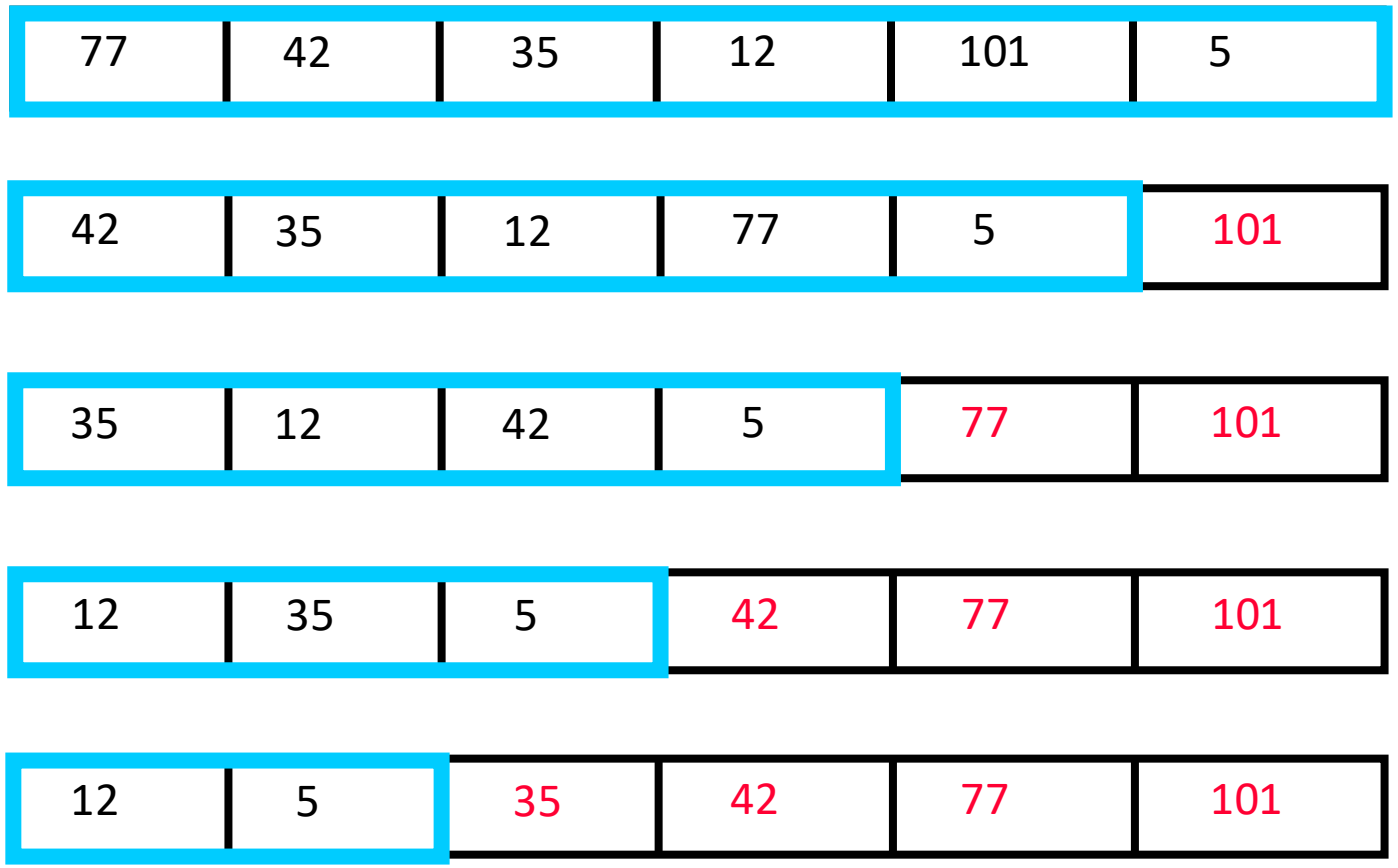
Repeat “Bubble Up” How Many Times?

- If we have N elements …
- And if each time we bubble an element, we place it in its correct location …
- Then we repeat the “bubble up” process N – 1 times
- This guarantees we’ll correctly place all N elements

“Bubbling” All the Elements



Reducing the Number of Comparisons

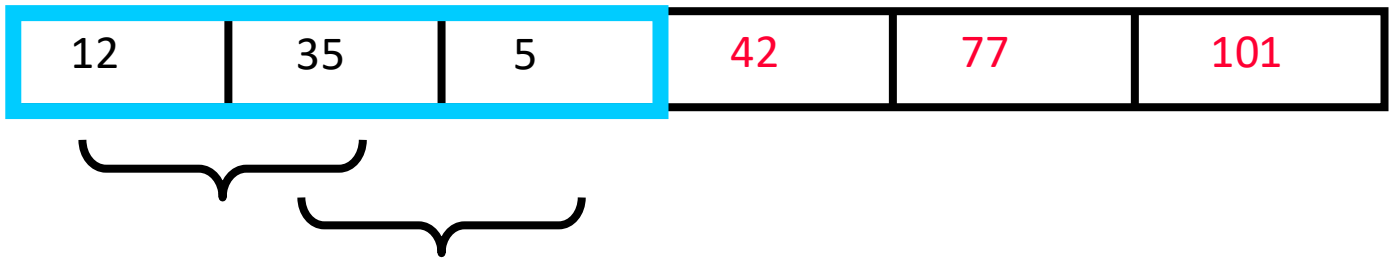


Reducing the Number of Comparisons

- On the i^{th} "bubble up", we only need to do MAX – i comparisons

For example:

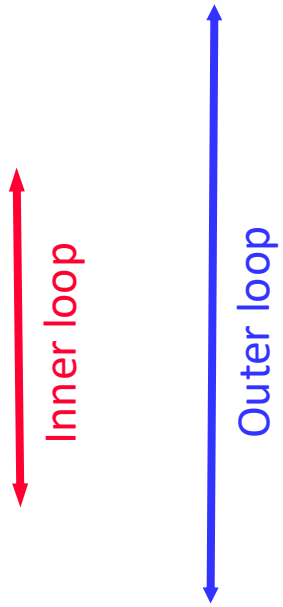
- This is the 4^{th} "bubble up"
- MAX is 6
- Thus we have 2 comparisons to do



Putting It All Together

```
algorithm Bubblesort(A)
  to_do, index isoftype Num
  to_do ← N - 1

  while to_do>0:
    index ← 1
    while index<to_do+1
      if(A[index] > A[index + 1]) then
        Swap(A[index], A[index + 1])
      endif
      index ← index + 1
    end while
    to_do ← to_do - 1
  end while
```



Already Sorted Collections?

- What if the collection was already sorted?
- What if only a few elements were out of place and after a couple of "bubble ups," the collection was sorted?
- We want to be able to detect this and "stop early"!

5	12	35	42	77	101

Using a Boolean “Flag”

- We can use a boolean variable to determine if any swapping occurred during the “bubble up”
- If no swapping occurred, then we know that the collection is already sorted!
- This boolean “flag” needs to be reset after each “bubble up”

Pseudo-Code

```
algorithm Bubblesort(A)
  to_do: index isoftype Num
  did_swap: Boolean
  to_do ← N – 1
  did_swap ← true

  while (to_do>0):
        index ← 1
         did_swap ← false
        while index<to_do+1
              if(A[index] > A[index + 1]) then
                    Swap(A[index], A[index + 1])
                    did_swap ← true
              endif
              index ← index + 1
        end while
        to_do ← to_do – 1
        if did_swap = false: break
end while
```

Which Loop Should We Analyze?

- Bubble sort has an inner loop and an outer loop
- The inner loop performs local comparisons
- The outer loop makes global progress
- Correctness is proved using the outer loop invariant

Think First: What Is Already Correct?

- Suppose k outer-loop iterations have completed
- Which elements will never move again?
- Which part of the array is already sorted?

Proving Bubble Sort (w.r.t. Outer Loop)


```
algorithm Bubblesort(A)
  to_do: index isoftype Num
  did_swap: Boolean
  to_do ← N – 1
  did_swap ← true

  while (to_do>0):
       index ← 1
        did_swap ← false
       while index<to_do+1
             if(A[index] > A[index + 1]) then
                   Swap(A[index], A[index + 1])
                   did_swap ← true
             endif
             index ← index + 1
       end while
       to_do ← to_do – 1
       if did_swap = false: break
end while
```

Proof of Correctness

Loop Invariant: A[to_do+1:N] is a sorted permutation of largest elements in final positions

1. Initialization

- Before the first iteration, an empty suffix is trivially sorted

2. Maintenance

- Inner loop moves the maximum of A[1..to_do+1] to position to_do+1; Then to_do is decremented→ Sorted suffix grows by one element

3. Termination

- Loop ends when to_do = 0 or no swaps occur. Invariant implies entire array A[1..N] is sorted. Correctness follows directly

Proving Insertion-sort

```
Algorithm InsertionSort(A)
Input: An array A with n elements
Output: A sorted in non-decreasing
        order

1. for i ← 2 to length(A) do
2.     key ← A[i]
3.     j ← i - 1
4.     while j > 0 and A[j] > key do
5.         A[j + 1] ← A[j]
6.         j ← j - 1
7.     end while
8.     A[j + 1] ← key
9. end for
10. return A
```

Proof of Correctness

Loop Invariant: A[1:i] is a sorted permutation of elements in the original A[1:i] at the end of iteration i

1. Initialization (Note: i ← 1 is not part of the loop)

- When i=1, A[1:1] contains one value → sorted.

2. Maintenance

- Within the loop-body, A[1:i-1] is assumed to be sorted. We move A[i-1], A[i-2], … toward the right, until we find a position for A[i]. Once A[i] is inserted, A[1:i] remains sorted.

3. Termination

- As i goes from 2 to length(A), and the loop body does not modify i. The loop will terminate. By termination, i=length(A) means A[1:length(A)] is sorted.

Common Trap (Think 30s)

香港科技大学(广州)
THE HONG KONG UNIVERSITY OF SCIENCE AND TECHNOLOGY (GUANGZHOU)

False statement

"The inner while loop sorts the array, so insertion sort is correct."

Your job

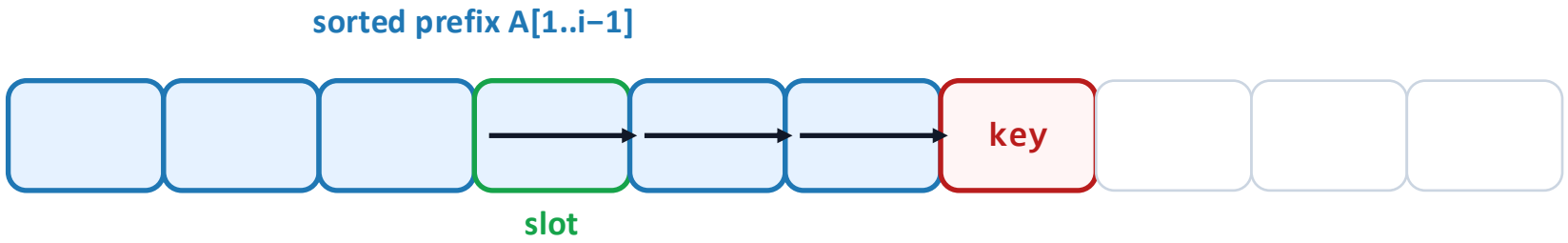
Which part of the array does the inner loop affect, and why is that not enough?

Rule of thumb

Correctness is proved by an invariant tied to the outer loop.

Role of the Inner Loop in Insertion Sort

- Shifts elements larger than key one position to the right
- Creates exactly one “slot” for key



Crucially:

- Preserves the relative order of the shifted block
- Does not disturb elements ≤ key

For a fully formal proof, an inner-loop invariant can help, but the outer-loop invariant is the main driver.

Case Study: Binary Search

Binary Search

Input

A sorted array A[1..n] (nondecreasing) and a target value x.

Goal

Return whether there exists an index k such that A[k] = x.

Binary search exploits sorted order to reduce the candidate region.

Binary Search

Find an element in a sorted array:

1. Check middle element.
2. Recursively search 1 subarray.

Binary Search

Find an element in a sorted array:

1. Check middle element.
2. Recursively search 1 subarray.

Example: Find 9

3	5	7	8	9	12	15

Binary Search

Find an element in a sorted array:

1. Check middle element.
2. Recursively search 1 subarray.

Example: Find 9

Binary Search

Find an element in a sorted array:

1. Check middle element.
2. Recursively search 1 subarray.

Example: Find 9

3 5 7 8 9 12 15

Binary Search

Find an element in a sorted array:

1. Check middle element.
2. Recursively search 1 subarray.

Example: Find 9

3 5 7 8 9 12 15

Binary Search

Find an element in a sorted array:

1. Check middle element.
2. Recursively search 1 subarray.

Example: Find 9

3 5 7 8 9 12 15

Binary Search

Find an element in a sorted array:

1. Check middle element.
2. Recursively search 1 subarray.

Example: Find 9

3 5 7 8 9 12 15

A Nearly Correct Binary Search


```
1 low = 1
2 high = n
3 while low < high:
4     mid = low + (high - low) // 2
5     if A[mid] < x:
6         low = mid
7     else:
8         high = mid
……
```

What is wrong with this algorithm?

A Concrete Failure Case

Let

$A = [1, 2], \quad x = 2$

Execution trace:

- low = 0, high = 1
- mid = 0
- $A[mid] < x \Rightarrow low \leftarrow mid = 0$

What happens?

The loop never makes progress.

How to Fix the Bug

Invariant we want

If x exists in A, then it lies in A[low..high].

Question: Which update guarantees progress when A[mid] < x?

- low ← mid
- low ← mid + 1

Explain

Why must at least one index be removed each iteration?

Binary Search

1) Preserve the invariant

If x exists, it remains in A[low..high].

2) Strictly shrink the interval (invariant)

At least one index is eliminated each iteration (so high−low decreases).

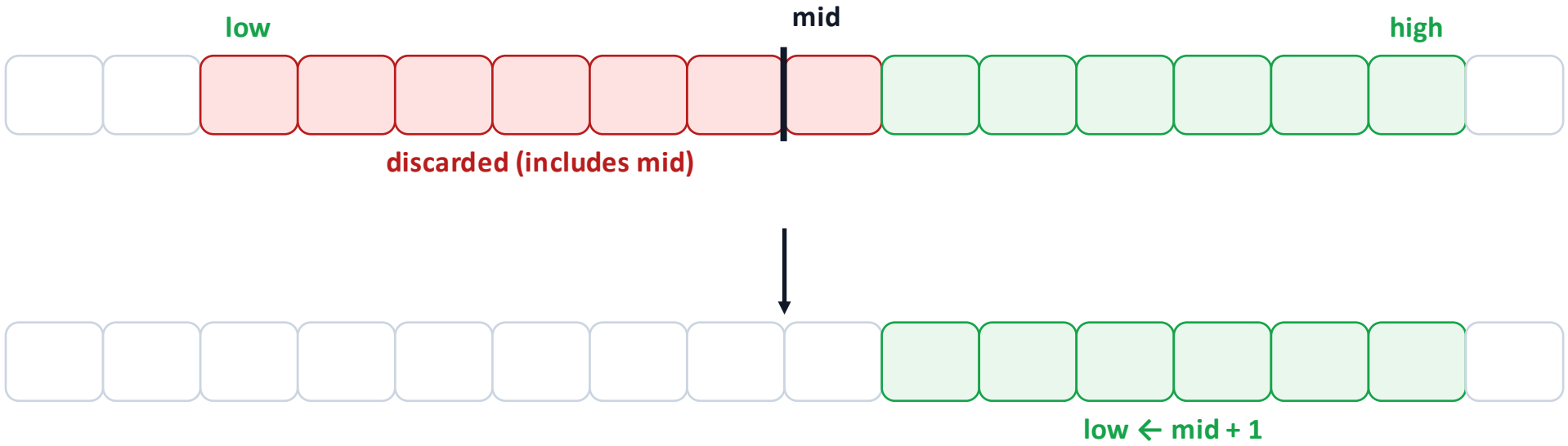
Invariant ⇒ partial correctness; shrinking ⇒ termination; together ⇒ correctness.

Correct Binary Search Algorithm

```
 1 low = 1
 2 high = n
 3 while low <= high:
 4     mid = low + (high - low) // 2
 5     if A[mid] == x:
 6         return true
 7     else if A[mid] < x:
 8         low = mid + 1
 9     else:
10         high = mid - 1
11 return false
```

Binary Search: The Interval Must Shrink

Invariant: if x is present, it lies in A[low..high].



We eliminate at least one index each iteration ⇒ termination.

Invariant

At the end of each iteration, if the target value x exists in A, then it must lie in the subarray A[low..high].

Case Study: KMP String Match

Scope: What We Do (and Do Not) Cover

- We use string matching only as an example to practice loop-invariant reasoning, especially two-index invariants involving (i, j).

- We do not teach the full KMP algorithm today.
  - The full algorithm will be introduced in Week 7.

String Matching

香港科技大学(广州)
THE HONG KONG UNIVERSITY OF SCIENCE AND TECHNOLOGY (GUANGZHOU)

Input: Text T[1..n] and pattern P[1..m].

Output: Decide whether P appears as a contiguous substring of T.

Naïve approach: try alignments and compare left-to-right. When a mismatch happens, it feels natural to "**shift** the pattern and try again".

How many comparisons are required? – O(___)

The key correctness question (not an efficiency question)

How do we know a shift does not skip a valid match?

Why Correctness Is Non-Trivial Here

- During matching, we repeatedly build a partial match and then break it by a mismatch.
- A careless shift can discard positions that still could be the start of a match.
- Testing cannot cover all texts/patterns; correctness must be a logical guarantee.

What do we need?

A statement that captures exactly what we know is already correct at any time. → *two-index loop invariant for this example*

State Definition: What Do i and j Mean?

- To state an invariant, we must define the state precisely:
  - i: current text position (the next text character to be processed/compared)
  - j: the number of pattern characters currently matched

- Meaning of "j matched": The prefix P[1..j] matches the suffix of already processed text:

$$P[1..j] = T[i-j..i-1].$$

(If j=0, the matched prefix is empty and the statement is trivially true.)

Loop Invariant for KMP

- Loop Invariant statement: At the end of every iteration boundary (with current **(i, j)**),P[1..j] matches a suffix of T[1..i−1].
  - The algorithm has already certified a length-j match ending at position i−1 in T.
  - All future updates must preserve this certified fact.

Match case: extending the certified/matched region

- Suppose we are about to compare T[i] with P[j+1], and they match. We then,
  - Increase i by 1.
  - Increase j by 1.
- Matching grows the invariant → The invariant still holds.

Mismatch: What Must Any "Jump" Preserve?

- Suppose we are about to compare T[i] with P[j+1], and they mismatch.
- Constraints we impose (the KMP philosophy, X.ref week 7):
  - We do not move the text pointer backward (i never decreases).
  - So the only freedom is to reduce j (try a shorter matched prefix).
    - But we are not allowed to reduce j arbitrarily.
    - The new value j′ must keep the invariant true. $P[1..j'] = T[i-j'..i-1]$.

Key message

A jump is safe iff it preserves the invariant. Any update violating the invariant is incorrect.

Do we skip a better alignment?

- When a mismatch occurs with j > 0, there may exist multiple values of 0< *j'* < j that satisfy the invariant $P[1..j'] = T[i - j' .. i - 1]$.

- Question: which j' should we choose?
  - We consider the largest possible j'
  - In other words, shift P **as little as possible** while keeping the invariant true.
  - This gives the **next possible starting position** for P in T, so **no possible match is skipped**.

  (*Will introduce how to choose j that satisfies this invariant in week 7.)

Full Proof

Loop Invariant:
At the end of each iteration (with current indices i, j), P[1..j] = T[i−j .. i−1]. That is, the prefix P[1..j] matches a suffix of the processed text T[1..i−1].

1. Initialization
Initially, i = 1 and j = 0. Both sides represent the empty string, so the invariant holds.

2. Maintenance
• If T[i] = P[j+1], both i and j increase by one, extending the certified match.
• If T[i] ≠ P[j+1], keep i fixed and choose the largest valid j (page 64).

3. Termination
If j = m, then by the invariant P[1..m] = T[i−m .. i−1], so a correct match is found.
If the text ends without reaching j = m, then no match exists.

Summary

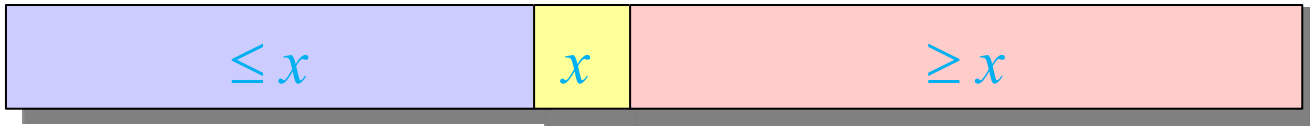
All our examples share the same structure: an invariant certifies a part of progress.

Algorithm	**Certified object (invariant)**
Max scan	max = max(A[1..i]) (certified prefix)
Insertion sort	A[1..i] is sorted (certified sorted prefix)
Bubble sort	A[to_do+1..n] sorted (certified sorted suffix)
String matching	P[1..j] = T[i-j..i-1] (certified match length)

Case Study: Quick Sort

Loop Invariant for Quick Sort

- A popular sorting algorithm discovered by C.A.R. Hoare in 1962
  - In many situations, it's the fastest, in $O(n \log n)$ time (for in-memory sorting)
- Basic scheme
  - Pivot selection: choose a number x from the input array as the pivot
  - Partition: partition an array into two subarrays around a pivot x such that elements in left subarray $\leq x \leq$ the elements in the right subarray



  - Recursion: recursively apply quicksort to each of the two subarrays

Quick Sort (Pseudo-Code)

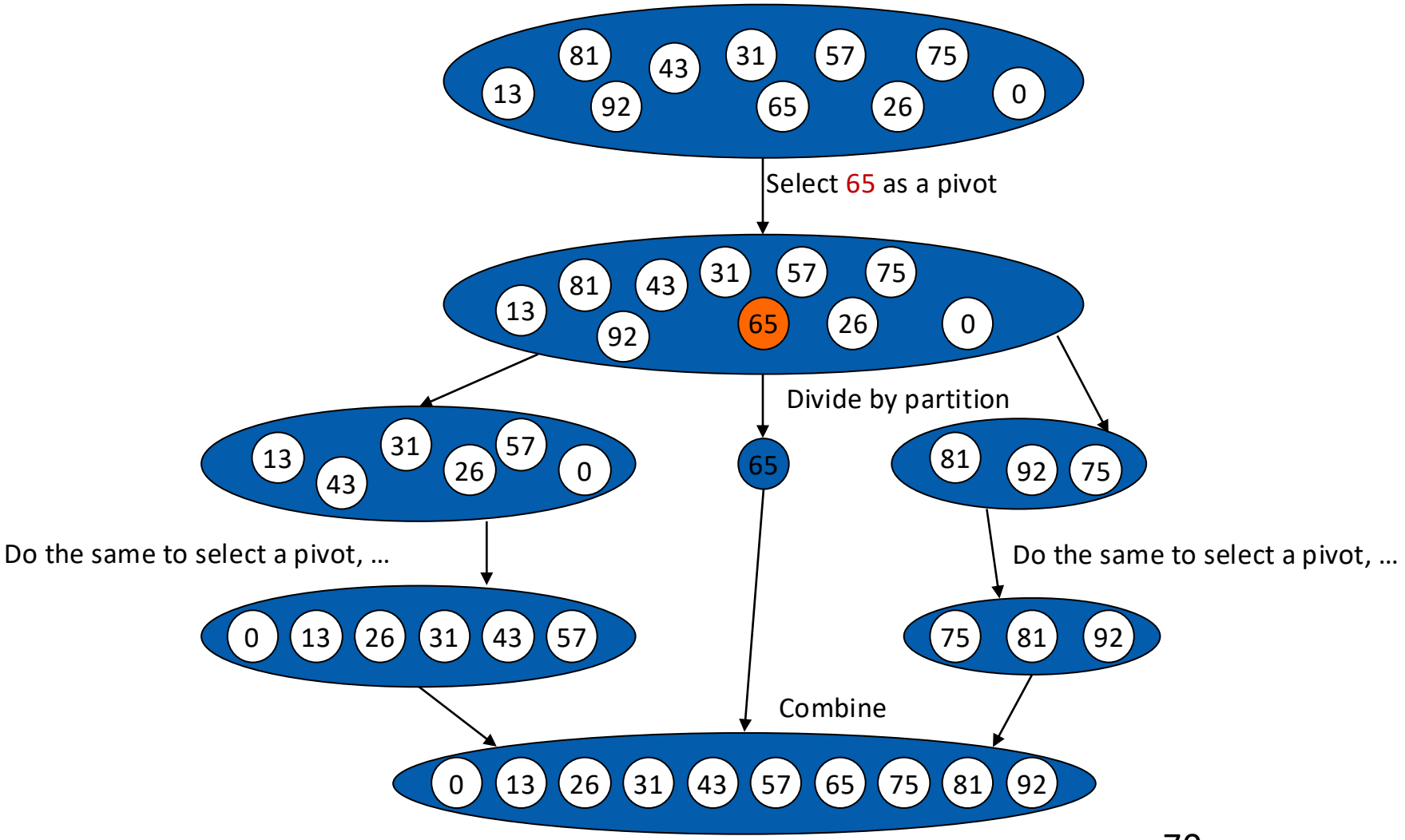
```
QUICKSORT(A, p, r)
    if p < r
        q ← PARTITION(A, p, r)
        QUICKSORT(A, p, q–1) //recursively sort the low side
        QUICKSORT(A, q+1, r) //recursively sort the high side
Initial call: QUICKSORT(A, 1, n)
```

Partition

Divide data into two groups, such that:

- All items in the left subarray are at most x
- All items in the right subarray are at least x

An Example


81
43
31
57
75
13
92
65
26
0
Select 65 as a pivot
81
43
31
57
75
13
92
65
26
0
Divide by partition
13
31
57
43
26
0
65
81
92
75
Do the same to select a pivot, …
Do the same to select a pivot, …
0
13
26
31
43
57
75
81
92
Combine
0
13
26
31
43
57
65
75
81
92

In-place Partitioning (Main Idea)

- The partition process (two indexs)
  - Suppose the rightmost element is the pivot
  - Start with two pointers: *leftIndex* initialized to one position to the left of the first cell; *rightIndex* to one position to the left of the last cell
  - *leftIndex* moves to the right; *rightIndex* moves to the left

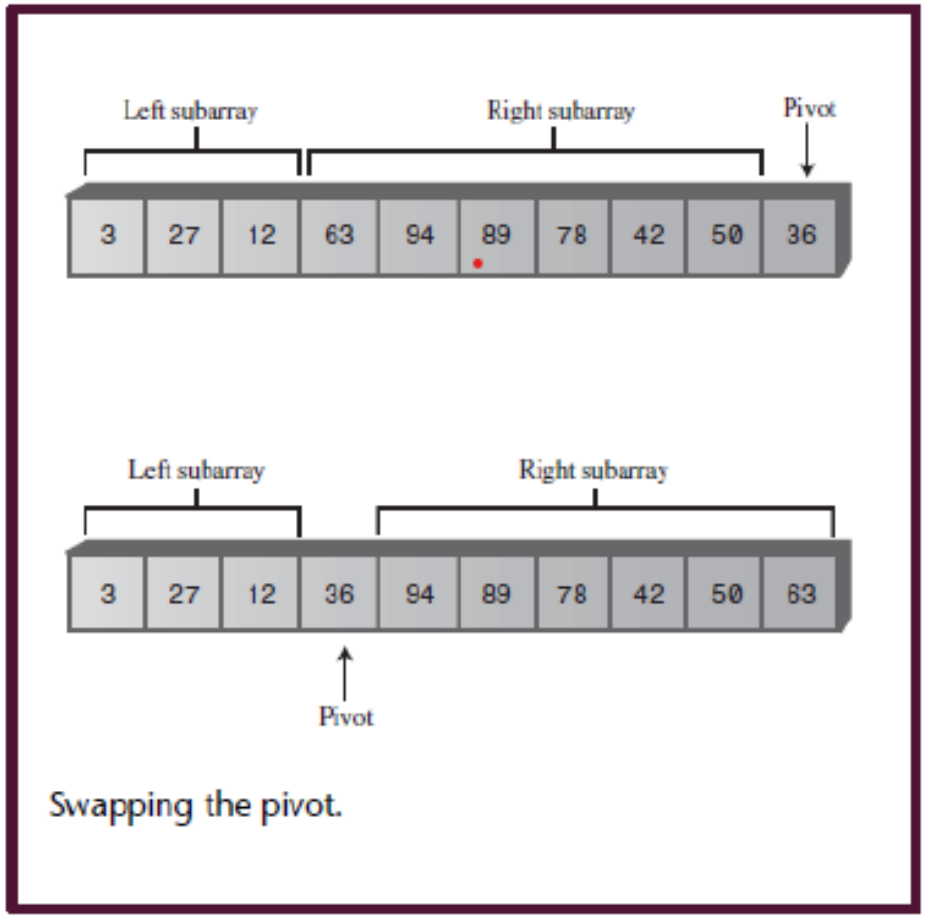
- Stopping and Swapping
  - Move *leftIndex* to the right until it finds a value **≥ pivot**.
  - Move *rightIndex* to the left until it finds a value **≤ pivot** or moves past the left boundary.
  - Stop if the two indices **meet or cross**.
  - Otherwise, **swap the two elements**, move both indices one step inward, and repeat.

Efficiency: Partitioning

- For an array of n elements:
  - left *leftIndex* starts at the first element and moves only to the **right**.
  - *rightIndex* starts just before the pivot and moves only to the **left**.
  - Each index moves at most n positions, so the total number of comparisons is **O(n)**.
  - The final swap takes **O(1)** time.
- The number of comparisons is O(n).

Partitioning (Details)

- Example:
  - **42 89 63 12 94 27 78 3 50 36**
- After the scan finishes, we have
  - **3 27 12 63 94 89 78 42 50 36**
- We swap the pivot with the leftmost element in the right subarray
  - **3 27 12 36 94 89 78 42 50 63**



Swapping the pivot.

Partition

```
Algorithm Partition(A, left, right):
    Input: Array A, starting index left, ending index right
    Output: Index of the pivot after rearrangement

    pivot ← A[right]   // Choose last element as pivot
    leftIndex ← left
    rightIndex ← right
    while true do:
        // Move leftIndex to the right until finding an element >= pivot
        while leftIndex ≤ right and A[leftIndex] < pivot do:
            leftIndex ← leftIndex + 1
        // Move rightIndex to the left until finding an element <= pivot
        while rightIndex ≥ left and A[rightIndex] > pivot do:
            rightIndex ← rightIndex - 1
        if leftIndex ≥ rightIndex then:
            break  // Indices have crossed, partitioning is complete
        swap A[leftIndex] and A[rightIndex]  // Swap elements
         leftIndex ++; rightIndex --
    swap A[right] and A[leftIndex]  // Move pivot to correct position
    return leftIndex  // Return final position of pivot
```

Think: How to prove the correctness of this algorithm?